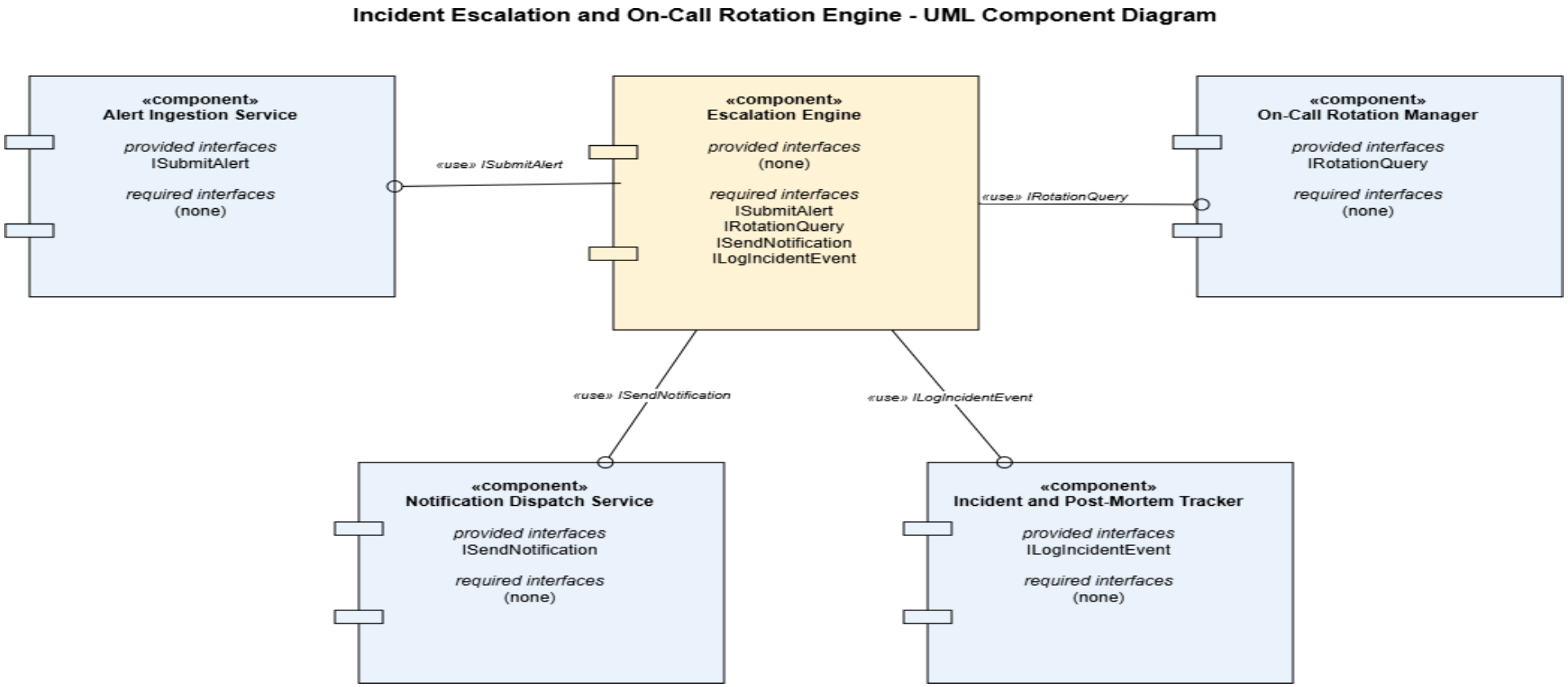
SE LAB 3

SRINIDHI P

PES1UG24AM407



Architectural Decision Justification

Architecture Selection

We chose Microservices Architecture for the Incident Escalation & On-Call Rotation Engine.

Reason 1

Incident response systems must remain reliable under partial failure: if the Notification Dispatch Service (SMS, phone, webhook, email) experiences a temporary outage, the Escalation Engine should still be able to log the incident and evaluate rotation schedules. Microservices architecture gives each function — alert ingestion, escalation logic, rotation management, notification dispatch, and post-mortem tracking — its own fault boundary, so one component failing does not cascade into a total system outage during a live incident.

Reason 2

Alert volume is highly bursty: a major outage can trigger dozens of simultaneous alerts, each requiring immediate multi-channel notification, while rotation schedule lookups and post-mortem writes remain comparatively low-traffic at the same moment. Microservices lets the Notification Dispatch Service scale independently during alert storms without over-provisioning the entire system, which would be unavoidable in a single-deployment layered design.

Security Advantage

Isolating the Notification Dispatch Service as its own deployable unit limits which part of the system holds third-party API credentials (SMS gateway keys, webhook secrets, email provider tokens). If this service is compromised, the blast radius is contained to notification delivery and does not expose the Escalation Engine's core incident logic or the On-Call Rotation Manager's schedule data.

Performance Benefit

The Alert Ingestion Service and Escalation Engine can respond to a new alert and begin the 5-minute escalation timer within the required latency window without waiting on the On-Call Rotation Manager to sync schedule changes or the Post-Mortem Tracker to finish writing historical records, since these run as independently deployed services communicating asynchronously rather than through shared in-process calls.